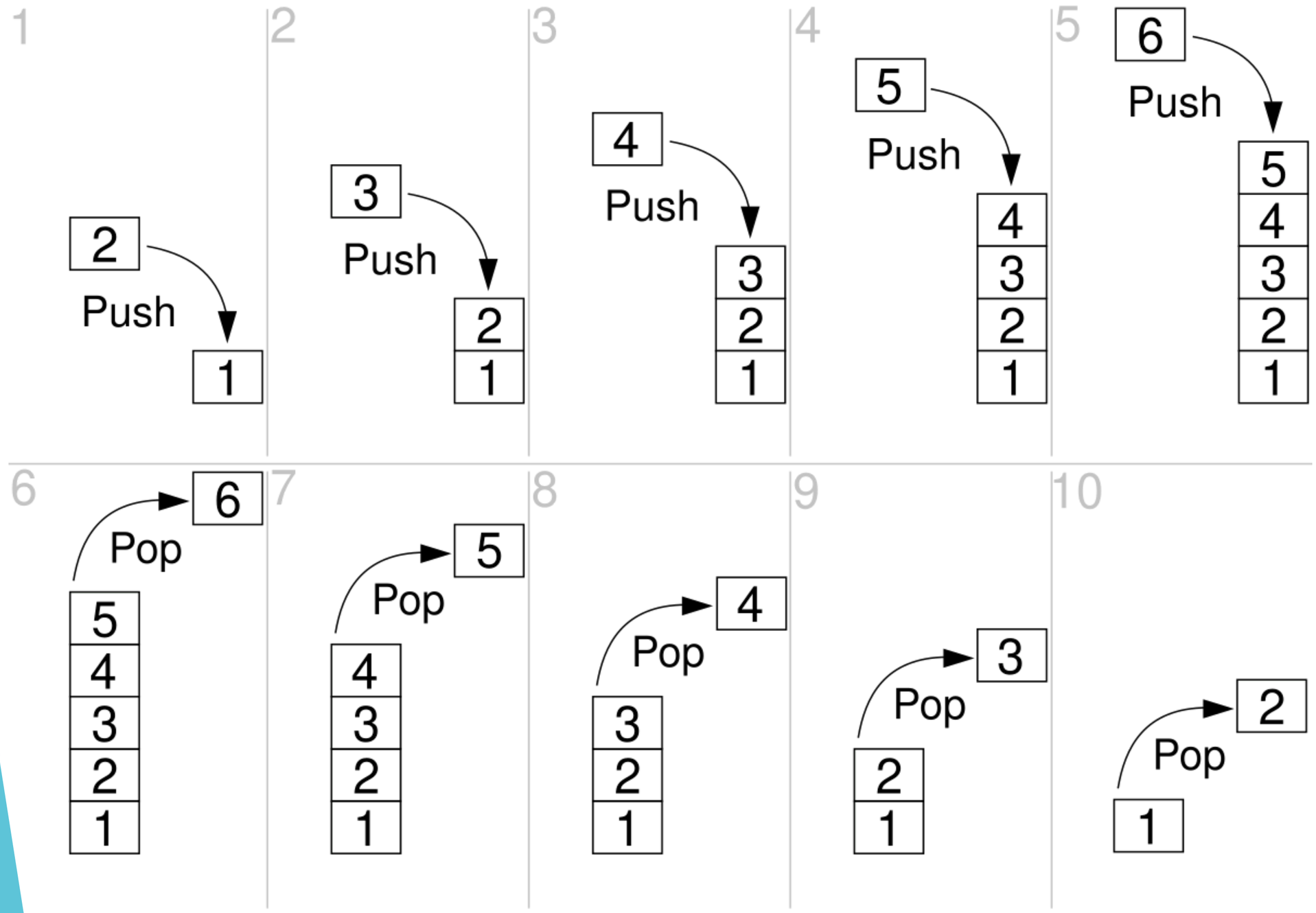
Python

PILHA

O que é PILHA ?

- Pense em uma pilha de pratos ou de livros. Você só pode adicionar um novo prato no topo da pilha, e quando for retirar um, você também retira o que está no topo. É impossível (ou muito desajeitado) tirar um prato do meio ou da base.
- Em ciência da computação, uma Pilha (Stack, em inglês) é exatamente isso: uma estrutura de dados linear que segue o princípio LIFO (Last-In, First-Out).

O último elemento adicionado à pilha será o primeiro a ser removido.


1
2
Push
1
2
3
Push
2
1
3
4
Push
3
2
1
4
5
Push
4
3
2
1
5
6
Push
5
4
3
2
1
6
6
Pop
5
4
3
2
1
7
5
Pop
4
3
2
1
8
4
Pop
3
2
1
9
3
Pop
2
1
10
2
Pop
1

Principais Operações de uma Pilha:

- **Push (Empilhar):** Adiciona um novo elemento ao topo da pilha. Método append
- **Pop (Desempilhar):** Remove e retorna o elemento que está no topo da pilha. Método pop
- **Peek / Top (Espiar):** Retorna o elemento do topo da pilha sem removê-lo. [-1]
- **isEmpty (Está Vazia?):** Verifica se a pilha não contém elementos. len(pilha)==0

Como Implementar Filas em Python?

- A maneira mais simples e natural de implementar uma pilha em Python é utilizando a estrutura de dados nativa list. As listas em Python são extremamente eficientes para operações no final da lista, o que se alinha perfeitamente com a lógica de uma pilha:
  - **Push:** Use o método **list.append(item)**. Ele adiciona um item ao final (topo) da lista.
  - **Pop:** Use o método **list.pop()**. Sem argumentos, ele remove e retorna o último item (topo) da lista.
- Ambas as operações (append e pop no final) são muito rápidas (complexidade de tempo O(1), em média), tornando a lista a escolha ideal para esta tarefa.

Exemplo 1

Inversor de Palavras

Problema

- Criar uma função que recebe uma string (uma palavra ou frase) e a retorna completamente invertida.

Conceito Aplicado:

- Este é o exemplo "clássico" para demonstrar o poder do LIFO.
- Vamos empurrar cada caractere da string para a pilha, um por um. O primeiro caractere ficará na base e o último no topo.
- Depois, vamos desempilhar os caracteres um por um.
- O primeiro a sair será o último que entrou, resultando na ordem invertida.

Arquivo
pilha.py

```
def inverter_string_com_pilha(texto):
    """
    Inverte uma string usando a lógica de uma pilha (implementada com uma lista).
    """
    pilha = [] # Nossa pilha será uma lista vazia

    # Fase 1: Empilhar (Push) cada caractere na pilha
    print(f"Empilhando os caracteres de '{texto}'...")
    for caractere in texto:
        pilha.append(caractere)
        print(f"  Push: {caractere} -> Pilha atual: {pilha}")

    # Fase 2: Desempilhar (Pop) para formar a string invertida
    string_invertida = ""
    print("\nDesempilhando para formar a nova string...")
    while pilha: # Enquanto a pilha não estiver vazia
        caractere_topo = pilha.pop()
        string_invertida += caractere_topo
        print(f"  Pop: {caractere_topo} -> String invertida: '{string_invertida}'")

    return string_invertida
```

Arquivo
main.py

```
# Executa a função
palavra_original = "Python"
palavra_invertida = inverter_string_com_pilha(palavra_original)

print("-" * 30)
print(f"Original: {palavra_original}")
print(f"Invertida: {palavra_invertida}")
```

Explicando

- O objetivo é criar uma função que pega uma string, como a palavra "Python", e a devolve escrita ao contrário, ou seja, "nohtyP". O interessante aqui não é apenas o resultado, mas como chegamos a ele usando a lógica de uma Pilha.
- Este exemplo é a demonstração perfeita do princípio LIFO (o último a entrar é o primeiro a sair).

Exemplo 2

Simulador de Navegação Web (Botão "Voltar")

Problema

- Simular o histórico de um navegador web. Conforme o usuário visita novaspáginas, elas são adicionadas ao histórico. Quando o usuário clica em "Voltar", ele retorna para a página visitada anteriormente

Conceito Aplicado

- Conceito Aplicado: A pilha é perfeita para gerenciar o histórico de "voltar".
- Cada nova página visitada é um push. Clicar em "Voltar" é um pop, que nosrevela a página anterior.

Arquivo navegador.py

```python
class Navegador:
    def __init__(self, pagina_inicial):
        # A pilha de histórico conterá as páginas visitadas
        self.historico = [pagina_inicial]
        self.pagina_atual = pagina_inicial
        print(f"Navegador iniciado. Página atual: {self.pagina_atual}")

    def visitar(self, url):
        """Simula a visita a uma nova página."""
        print(f"Visitando página: {url}")
        self.historico.append(url) # Push: adiciona a nova página ao topo do histórico
        self.pagina_atual = url
        self.mostrar_estado_atual()
  def voltar(self):
        """Simula o clique no botão 'Voltar'."""
        if len(self.historico) > 1:
            # Primeiro, desempilhamos a página atual
            self.historico.pop() # Pop
            # A nova página atual é a que está no topo da pilha agora
            self.pagina_atual = self.historico[-1] # Peek (espiar o topo)
            print(f"--> Voltando para: {self.pagina_atual}")
        else:
            print("--> Não há mais histórico para voltar.")
        self.mostrar_estado_atual()

    def mostrar_estado_atual(self):
        print(f"  Página Atual: {self.pagina_atual}")
        print(f"  Histórico (Pilha): {self.historico}\n")
```

Arquivo
main.py

```
# Simulação de uso
meu_navegador = Navegador("google.com")
meu_navegador.visitar("wikipedia.org")
meu_navegador.visitar("youtube.com")

meu_navegador.voltar()
meu_navegador.voltar()
meu_navegador.voltar() # Tenta voltar além do início
```

Resumo e Próximos Passos

- **Entenda o Conceito:** Solidifique o princípio **LIFO (Last-In, First-Out)**. É o oposto de uma fila.
- **Use list:** Para pilhas em Python, a lista padrão é a ferramenta perfeita e mais eficiente. Use **append()** para empilhar e **pop()** para desempilhar.
- **Pratique:** A pilha é fundamental em algoritmos como a **Busca em Profundidade (DFS - Depth-First Search)**, recursão, avaliação de expressões matemáticas e muito mais. Tente entender como a pilha de chamadas de funções funciona em Python, pois ela usa exatamente essa lógica!

Fim

Maromo